

FACTORY PATTERN

Creational Design Pattern

Factory Pattern

Imagine you're building a **food ordering app** where customers can order different types of meals (**Pizza, Burger, Pasta**). Initially, you might create separate classes for each food type and create objects like this:

```
pizza = Pizza()  
burger = Burger()  
pasta = Pasta()
```

But as your restaurant grows and you keep adding new menu items (**Biryani, Dosa, Momos, Chinese, etc.**), directly creating these objects in your code becomes messy and hard to manage, especially when the creation logic gets more complex.

Problems

The client code (**RestaurantService**) is tightly connected to specific food classes (Pizza, Burger, Pasta), making it dependent on knowing about each food type.

Adding new food items means you have to go back and modify the **RestaurantService** code, breaking the Open/Closed Principle.

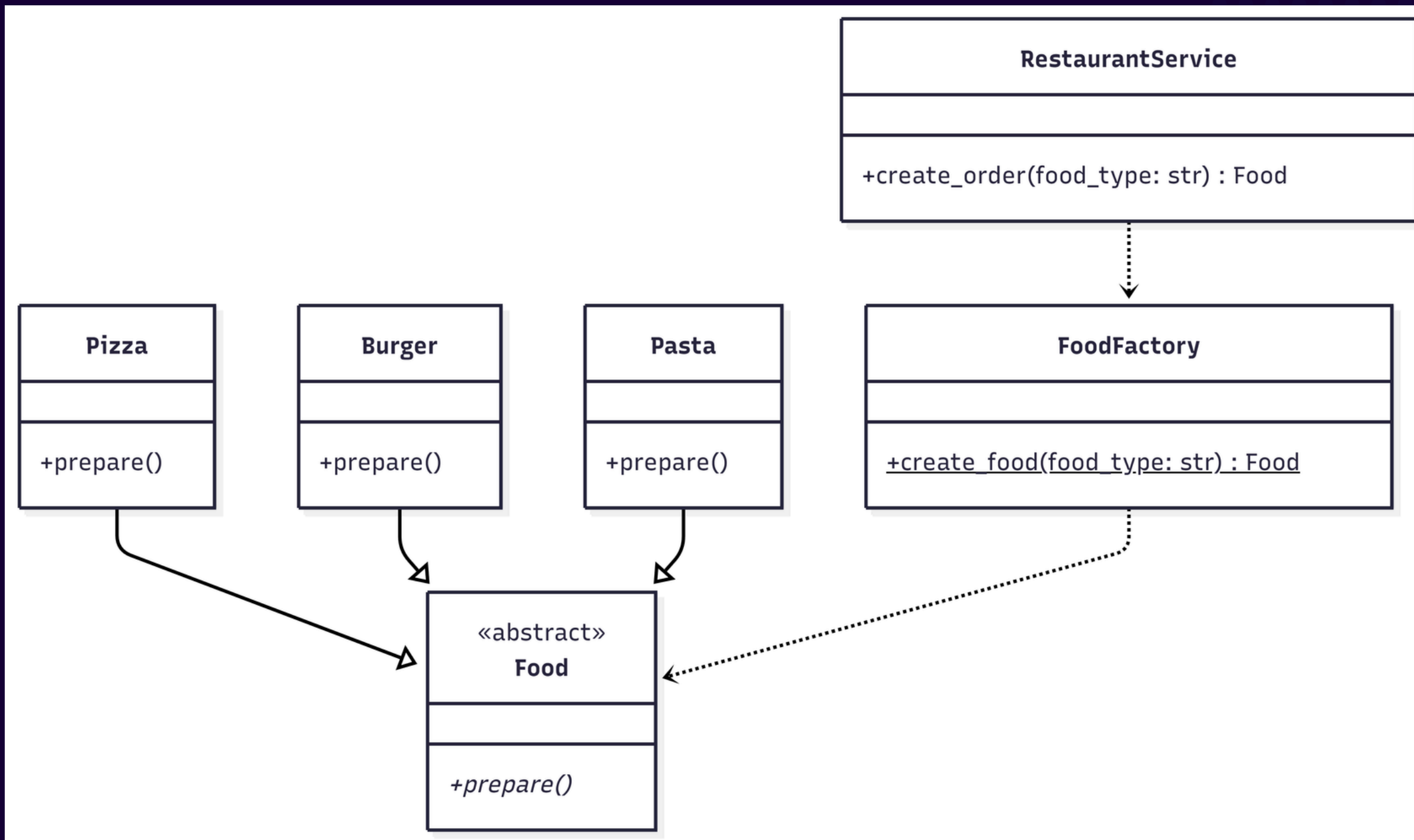
Solution

The **Factory Pattern** moves all **object creation** logic into one central place – the factory class. Instead of creating objects directly throughout your code, you hand over this responsibility to the factory, which decides which specific class to create based on the request. This approach follows the **Open/Closed Principle** by allowing you to add new food types without changing any existing code – you only **update the factory**.

Factory Pattern

1. **Factory Class:** The **FoodFactory** class holds all the logic for creating different food items based on what the customer orders. This keeps all creation logic in one place, making it simple to add or modify food types.
2. **Decoupling:** The **RestaurantService class (client)** no longer needs to know how Pizza, Burger, or Pasta objects are created. It simply asks the factory to create the food item and receives it ready to use.
3. **Flexibility:** Adding a new food type (like Biryani) only means updating the factory – the **RestaurantService** code stays completely unchanged.

UML Diagram



Real World Use Cases

1. **User Interface Libraries:** When building apps, the type of button or UI element created depends on which platform you're running on (Windows, macOS, or Linux). A factory decides which platform-specific component to create at runtime.
2. **Database Connections:** When an application needs to connect to different database systems (like MySQL or MongoDB), a factory chooses which database connector to create based on the configuration settings.
3. **File Export Tools:** When users want to export documents to different formats (PDF, Word, HTML), a factory determines which file generator to create based on the user's selection.